Contents

1. Import Libraries + Setup....2
1.5 GPU Device Setup....2
2. Data Load + EDA....2
Visualization....2
3. Preprocessing....3
Hidden Missing Value Handle (if Need)....3
3.2 Feature / Target Alada + Train / Val / Test Split....3
3.3 Scaling....3
3.4 Label Encoding (if the target column categorical/text based)....3
4. NumPy → PyTorch Tensor & DataLoader....4
5. Model Architecture (nn.Module, MLP)....4
Model Summary Dekhte (torchinfo)....5
6. Loss Function ও Optimizer....5
Rule: Kon Loss Function kokhon....5
→Binary Classification: Use nn.BCEWithLogitsLoss() (no activation) or nn.BCELoss() with a final Sigmoid layer.5
→Multi-class Classification: Use nn.CrossEntropyLoss() with raw logits (no activation needed as Softmax is built-in)....5
→Regression: Use nn.MSELoss() with raw outputs (no activation needed).....5
7. Complete Training Loop....5
8. Training Curve Visualization....6
9. Test Set Evaluation....7
10. Confusion Matrix....7
11. ROC Curve (Binary Classification-e)....7
12. Model Save....7
14. Rule To Remember (DL Project)....8

Deep Learning Full Project Cheat Note

1. Import Libraries + Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.decomposition import PCA
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, roc_curve, auc, classification_report
)

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader

SEED = 42                    # reproducibility - protibar shomoy same result pete
torch.manual_seed(SEED)
np.random.seed(SEED)
```

Note: seed fix na korle protibar notebook run korle alada alada result asbe — debug/compare kora kothin hoye jay।

1.5 GPU Device Setup

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

2. Data Load + EDA

```
df = pd.read_csv("data.csv")
df.head()
df.shape
df.dtypes
df.isnull().sum()                    # missing value ache kina check
df.describe()                        # scale koto vinno seta bujhte (scaling lagbe kina)
df['Target'].value_counts()          # class balanced na imbalanced check
df['Target'].value_counts(normalize=True) * 100   # percentage
```

Visualization

```
# Class distribution
df['Target'].value_counts().plot(kind='bar', color=['steelblue','salmon'])
plt.title("Class Distribution"); plt.show()

# Feature distribution (kono feature skewed/outlier ache kina bujhte)
df.drop(columns=['Target']).hist(figsize=(12,10), bins=20)
plt.tight_layout(); plt.show()
```

```
# Correlation heatmap - kon feature target er shathe beshi related
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Correlation Heatmap"); plt.show()
```

3. Preprocessing

Hidden Missing Value Handle (if Need)

Kono column-e 0 value 'medically/logically impossible' hole (jemon Glucose=0, BMI=0), seta actually missing data median diye replace kora hoy.

```
cols_with_hidden_missing = ["Glucose", "BloodPressure", "BMI"]   # dataset onujayi column bosabe
df_clean = df.copy()
for col in cols_with_hidden_missing:
    median_val = df_clean[df_clean[col] != 0][col].median()   # 0 baad diye median ber kora
    df_clean[col] = df_clean[col].replace(0, median_val)
```

3.2 Feature / Target Alada + Train / Val / Test Split

★ 70/15/15 split (train/val/test)

```
X = df_clean.drop(columns=['Target']).values
y = df_clean['Target'].values

# stratify=y dile protita split e class proportion shoman thakbe
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=SEED, stratify=y
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=SEED, stratify=y_temp
)
```

3.3 Scaling

★ The scaler should be fitted (using `fit_transform`) only on the training data, while for validation/test data, only `transform` should be applied otherwise, data leakage occurs.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
```

3.4 Label Encoding (if the target column categorical/text based)

```
encoder = LabelEncoder()
y_train = encoder.fit_transform(y_train)
y_test = encoder.transform(y_test)
```

4. NumPy → PyTorch Tensor & DataLoader

★

```
X_train_t = torch.tensor(X_train_scaled, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)   # [N] na hoye [N,1] shape (BCELoss er jonno)

X_val_t = torch.tensor(X_val_scaled, dtype=torch.float32)
y_val_t = torch.tensor(y_val, dtype=torch.float32).reshape(-1, 1)

X_test_t = torch.tensor(X_test_scaled, dtype=torch.float32)
y_test_t = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)

# TensorDataset - X o y jora bendhe akta dataset object
train_dataset = TensorDataset(X_train_t, y_train_t)
val_dataset = TensorDataset(X_val_t, y_val_t)

# DataLoader - dataset ke batch e vag kore model e pathanor jonno
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)   # train e shuffle=True
val_loader = DataLoader(val_dataset, batch_size=16, shuffle=False)
```

5. Model Architecture (nn.Module, MLP)

★ Ei class template shob project e reuse hobe - shudhu layer size/count change korbe

```
class MyMLP(nn.Module):

    #Input(n feature) -> Hidden1(16) -> Hidden2(8) -> Output(1, probability)

    def __init__(self, input_size):
        super().__init__()   # nn.Module er parent __init__ call kora badhyotamulok

        self.layer1 = nn.Linear(input_size, 16)
        self.layer2 = nn.Linear(16, 8)
        self.output_layer = nn.Linear(8, 1)

        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()
        self.dropout = nn.Dropout(0.2)          # training e randomly 20% neuron off - overfitting komay

    def forward(self, x):
        x = self.relu(self.layer1(x))
        x = self.dropout(x)
        x = self.relu(self.layer2(x))
        x = self.dropout(x)
        x = self.sigmoid(self.output_layer(x))
        return x

model = MyMLP(input_size=X_train.shape[1])
print(model)

total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print("Total trainable parameters:", total_params)
```

Model Summary Dekhte (torchinfo)

```
!pip install torchinfo
from torchinfo import summary

summary(model, input_size=(batch_size, num_features))   # protita layer er shape+param count dekhabe
```

6. Loss Function ৩ Optimizer

```
criterion = nn.BCELoss()      # Binary classification (model e already Sigmoid ache tai)

optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=0.0001)

EPOCHS = 150
```

Rule: Kon Loss Function kokhon

→Binary Classification: Use nn.BCEWithLogitsLoss() (no activation) or nn.BCELoss() with a final Sigmoid layer.

→Multi-class Classification: Use nn.CrossEntropyLoss() with raw logits (no activation needed as Softmax is built-in).

→Regression: Use nn.MSELoss() with raw outputs (no activation needed).

7. Complete Training Loop

★

```
train_losses, val_losses, val_accuracies = [], [], []

for epoch in range(1, EPOCHS + 1):

    model.train()
```

```
    running_loss = 0.0

    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()
        prediction = model(X_batch)         # forward propagation
        loss = criterion(prediction, y_batch)
        loss.backward()                     # backward propagation (gradient ber kora)
        optimizer.step()                    # gradient onujayi weight/bias update

        running_loss += loss.item() * X_batch.size(0)

    epoch_train_loss = running_loss / len(train_dataset)
    train_losses.append(epoch_train_loss)

    # ---------------- Validation phase ----------------
    model.eval()                            # dropout bondho
    val_running_loss = 0.0
    correct = 0

    with torch.no_grad():                   # gradient hisab bondho (val/test e lagbe na)
        for X_batch, y_batch in val_loader:
            prediction = model(X_batch)
            loss = criterion(prediction, y_batch)
            val_running_loss += loss.item() * X_batch.size(0)

            predicted_class = (prediction >= 0.5).float()   # threshold diye probability -> class
            correct += (predicted_class == y_batch).sum().item()

    epoch_val_loss = val_running_loss / len(val_dataset)
    epoch_val_acc = correct / len(val_dataset)
    val_losses.append(epoch_val_loss)
    val_accuracies.append(epoch_val_acc)

    if epoch % 15 == 0 or epoch == 1:
        print(f"Epoch {epoch}/{EPOCHS} | Train Loss: {epoch_train_loss:.4f} | "
              f"Val Loss: {epoch_val_loss:.4f} | Val Acc: {epoch_val_acc*100:.2f}%")
```

⚠ optimizer.zero_grad() bhule gele gradient accumulate hoye jay ebong model thik moto train hoy na — kokhono baad debe na।

8. Training Curve Visualization

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(train_losses, label="Train Loss")
axes[0].plot(val_losses, label="Validation Loss")
axes[0].set_xlabel("Epoch"); axes[0].set_ylabel("Loss")
axes[0].set_title("Loss over Epochs"); axes[0].legend()
```

```
axes[1].plot(val_accuracies, color='green', label="Validation Accuracy")
axes[1].set_xlabel("Epoch"); axes[1].set_ylabel("Accuracy")
axes[1].set_title("Validation Accuracy over Epochs"); axes[1].legend()

plt.tight_layout(); plt.show()
```

Note: Train loss komchhe kintu val loss barchhe hole eita overfitting-er lokkhon।

9. Test Set Evaluation

★

```
model.eval()
with torch.no_grad():
    test_probabilities = model(X_test_t)
    test_predictions = (test_probabilities >= 0.5).float()

y_true_np = y_test_t.numpy()
y_pred_np = test_predictions.numpy()
y_prob_np = test_probabilities.numpy()

print(f"Accuracy : {accuracy_score(y_true_np, y_pred_np)*100:.2f}%")
print(f"Precision: {precision_score(y_true_np, y_pred_np, zero_division=0)*100:.2f}%")
print(f"Recall   : {recall_score(y_true_np, y_pred_np, zero_division=0)*100:.2f}%")
print(f"F1 Score : {f1_score(y_true_np, y_pred_np, zero_division=0)*100:.2f}%")
```

10. Confusion Matrix

```
cm = confusion_matrix(y_true_np, y_pred_np)

plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Class 0','Class 1'], yticklabels=['Class 0','Class 1'])
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.title("Confusion Matrix"); plt.show()
```

11. ROC Curve (Binary Classification-e)

```
fpr, tpr, thresholds = roc_curve(y_true_np, y_prob_np)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, color='darkorange', label=f"ROC curve (AUC = {roc_auc:.3f})")
plt.plot([0, 1], [0, 1], color='gray', linestyle='--', label="Random guess")
plt.xlabel("False Positive Rate"); plt.ylabel("True Positive Rate")
plt.title("ROC Curve"); plt.legend(); plt.show()
```

12. Model Save

★

```
torch.save(model.state_dict(), "model.pth")     # shudhu trained weight save hoy

import joblib
```

```
joblib.dump(scaler, "scaler.pkl")              # scaler o save kora lagbe -
                                               # notun data e prediction korar shomoy
                                               # same scaling apply korte hobe
# inference er age eval() call kora must
```

14. Rule To Remember (DL Project)

- Scaling শুধু train data দিয়ে fit করবে (fit_transform), val/test-এ শুধু transform — data leakage এড়াতে

- dtype=torch.float32 সবসময় ব্যবহার করবে (PyTorch default)

- Binary target hole reshape(-1,1) kore [N,1] shape banabe (BCELoss er jonno)

- model.train() → training e (dropout active), model.eval() → validation/test e (dropout off)

- torch.no_grad() → validation/test/inference e always use korbe (gradient dorkar nai, speed/memory bachbe)

- optimizer.zero_grad() → loss.backward() → optimizer.step() — ei 3 line order shobshomoy same

- Class imbalance thakle shudhu accuracy na, precision/recall/F1/confusion matrix dekhte hobe

- Binary → BCELoss+Sigmoid | Multi-class → CrossEntropyLoss+raw logits | Regression → MSELoss

- GPU use korle model ebong protita batch data (X, y) dutokei .to(device) diye pathate hobe